

DocuMind

Current Project — Backend Structure Detailed Folder, File & Responsibility Reference

Backend only • current implementation • no frontend flow • no GitHub workflow

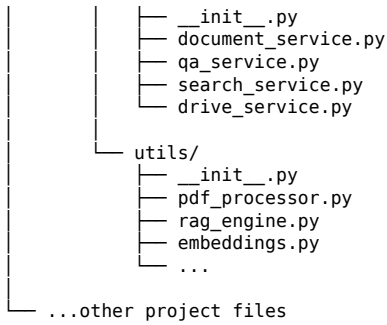
1. Scope of This Document

This document describes the backend structure of the current DocuMind project. It focuses only on the backend code, its folders/files, responsibilities and current RAG execution path.

- Django + Django REST Framework backend.
- PostgreSQL + pgvector persistence and vector search.
- Celery background document processing.
- Redis as Celery broker/cache infrastructure.
- PDF, DOCX and TXT extraction.
- Current fixed word-based chunking.
- Sentence Transformer embeddings.
- Similarity retrieval and source chunks.
- Groq-based answer generation.
- Chat history and document ownership/authorization.
- Google Drive integration.
- Current backend limitations compared with the planned advanced architecture.

2. Current Backend Project Structure

```
documind/
├── manage.py
├── requirements.txt
├── .env
├── .gitignore
├── README.md
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   ├── wsgi.py
│   └── celery.py
├── apps/
│   └── rag/
│       ├── __init__.py
│       ├── admin.py
│       ├── apps.py
│       ├── models.py
│       ├── serializers.py
│       ├── urls.py
│       ├── views.py
│       ├── permissions.py
│       ├── tasks.py
│       └── services/
```



3. Root Backend Files

manage.py

Django's command-line entry point. Used for running the development server, migrations, shell, tests and other Django management commands.

requirements.txt

Backend Python dependencies such as Django/DRF, database libraries, Celery, Redis, PyMuPDF, python-docx, Sentence Transformers and LLM/provider packages.

.env

Environment-specific secrets and configuration such as database, Redis and provider credentials. It should not be committed.

.gitignore

Prevents secrets, virtual environments, caches and other local/generated files from entering version control.

README.md

Backend/project documentation, setup instructions, architecture overview and usage information.

4. config/ — Django Project Configuration

- `__init__.py` — marks the configuration directory as a Python package.
- `settings.py` — central Django configuration: installed apps, database, REST framework, authentication, media/static settings and environment configuration.
- `urls.py` — top-level URL router that connects project URLs to application endpoints.
- `asgi.py` — ASGI entry point for deployment and async-capable serving.
- `wsgi.py` — WSGI entry point for traditional deployment.
- `celery.py` — initializes/configures Celery so Django can execute background tasks.

5. apps/rag/ — Current Main Backend Application

The current project keeps the core document/RAG functionality inside the Django RAG application. This app is responsible for API endpoints, document processing orchestration, search, question answering and Drive-related operations.

models.py — Database/Data Layer

Defines the persistent data structures used by the current RAG application, including documents, document chunks, chat/history-related records and processing metadata.

- Stores document ownership so users can access only their own permitted documents.
- Stores processed document chunks.

- Stores chunk metadata such as page number, chunk index and size.
- Supports vector-related data used by pgvector.
- Provides the database foundation for document search and chat history.

serializers.py — API Data Validation

Django REST Framework serializers validate incoming API data and convert backend models into API responses.

- Validates document upload/request data.
- Serializes document information for API clients.
- Serializes question-answer/search related response data where applicable.

views.py — API Endpoint Layer

views.py receives HTTP requests, checks permissions/validation and delegates actual business work to services.

- DocumentListView — authenticated document listing and upload.
- Document deletion/status/history endpoints.
- QuestionAnswer.post — receives a user question and calls QAService.answer_question.
- SearchView — calls SearchService for search/browse functionality.
- Google Drive connect/disconnect/sync endpoints.
- Uses IsAuthenticatedAndVerified for protected operations.

urls.py — Application Routes

Connects the RAG application's API URLs to the corresponding Django REST Framework views.

permissions.py — Access Control

Controls whether an authenticated user is allowed to access application resources. Document ownership is an important security boundary.

tasks.py — Background Processing

Contains the current Celery document-processing task.

- process_document_task receives a document ID.
- Locks/checks the document before processing.
- Skips processing if the document is already processed.
- Records processing start state.
- Calls the document processing service.
- Marks the document processed after success.
- Stores processing errors when failures occur.
- Retries failed processing up to three times with a delay.
- Handles documents that are deleted while processing.
- Cleans up chunks when appropriate.

admin.py / apps.py / __init__.py

- admin.py — Django admin registrations/configuration.
- apps.py — Django application configuration.

- `__init__.py` — package marker.

6. apps/rag/services/ — Current Business Logic

The service layer keeps larger application operations outside the HTTP views. This is one of the important architectural boundaries already present in the current backend.

document_service.py

Responsible for document-level business operations and processing orchestration.

- Receives a document upload/application request.
- Creates or prepares the document record.
- Starts or connects document processing to Celery.
- Coordinates document lifecycle operations.

qa_service.py

This is the current question-answering orchestration layer.

```

Question
↓
Check document ownership
↓
Check document processed state
↓
Load chat history
↓
search_similar_chunks(...)
↓
If no chunks → "Answer not found in document"
↓
build_context(...)
↓
build_prompt(...)
↓
generate_answer(...)
↓
calculate_confidence(...)
↓
Save chat history
↓
Return answer + sources + confidence

```

- Checks that the requested document belongs to/ is accessible by the user.
- Checks that processing has completed.
- Loads previous chat history in creation order.
- Retrieves `top_k=3` similar chunks.
- Builds a context containing retrieved sources and page information.
- Generates the answer through the RAG engine.
- Calculates a confidence score from retrieval similarity scores.
- Persists chat history safely.
- Returns question, answer, source chunks, document name and confidence score.

search_service.py

Provides the current search/browse application behavior. It acts as the service boundary between API views and lower-level search/retrieval utilities.

drive_service.py

Coordinates Google Drive related application operations such as connection/synchronization and document ingestion from Drive.

7. apps/rag/utils/ — Current RAG Processing Utilities

pdf_processor.py — Current Ingestion + Chunking

This is the main current document-processing utility.

- PDF extraction uses PyMuPDF (fitz), page by page.
- TXT files are extracted as text.
- DOCX files are extracted with python-docx, currently focusing on paragraphs.
- `clean_text()` normalizes whitespace/newlines and removes unsupported characters.
- `create_chunks()` currently uses fixed word-based chunking.
- `process_document()` currently calls `create_chunks` with `chunk_size=100` and `chunk_overlap=50`.
- Each chunk stores text, page number, chunk index and size.
- Old chunks are deleted before rebuilding.
- New chunks are inserted in bulk.
- Embedding storage is then triggered through `store_document_chunks(document.id)`.

Current Chunking Logic

```
Document text
  ↓
Clean text
  ↓
Fixed word window
  ↓
100-word chunk
  ↓
50-word overlap
  ↓
Next 100-word chunk
  ↓
Repeat
```

Important: the current implementation is not semantic chunking. It does not yet understand section meaning, headings, table boundaries or semantic topic boundaries.

rag_engine.py — Current LLM/RAG Generation

Handles context construction, prompt creation, LLM generation and the current confidence calculation.

- Uses Groq for answer generation.
- Default model in the current implementation is `openai/gpt-oss-20b`.
- Retrieved chunks are labeled as sources with page information.
- Conversation history is used to resolve references/context.
- The prompt instructs the model to answer only from the supplied document context.
- Current generation uses low temperature and a bounded token output.
- `calculate_confidence()` currently averages retrieved similarity scores.

Important: the current confidence score is a retrieval-similarity heuristic. It is not a true groundedness verifier.

embeddings.py / embedding utility

The current embedding layer uses Sentence Transformers, with the project using the `all-MiniLM-L6-v2` embedding model. Embeddings are stored/queryable through the PostgreSQL + pgvector setup.

8. Current Database / Search Backend

Component	Current responsibility
PostgreSQL	Persistent application/document/chat data.
pgvector	Stores document chunk embeddings and supports vector similarity search.
Document chunks	Contain chunk text plus page/index/size metadata.
Similarity search	Finds chunks semantically similar to the user's question.
User ownership	Restricts document operations to authorized users.

9. Current Document Processing Flow

```

Upload document
↓
Django API
↓
DocumentService
↓
Celery process_document_task
↓
process_document()
↓
Detect file type
├── PDF → PyMuPDF
├── DOCX → python-docx
└── TXT → text read
↓
clean_text()
↓
fixed word-based chunking
(chunk_size=100, overlap=50)
↓
store chunk text + metadata
↓
generate/store embeddings
↓
PostgreSQL + pgvector
↓
Document ready for retrieval

```

10. Current Question Answering Flow

```

User question
↓
QuestionAnswer API
↓
QAService.answer_question()
↓
Ownership / processed check
↓
Load chat history
↓
Vector similarity search
↓
Top 3 relevant chunks
↓
Build context
↓
Build prompt
↓
Groq LLM
↓
Answer
↓
Similarity-based confidence
↓
Save chat history
↓
Return answer + sources + confidence

```

11. Current Search Flow

```

Search request
↓
SearchView
↓
SearchService
↓

```

```
Search/retrieval utility
↓
Filter by document/user access
↓
Similarity results
↓
Return matching chunks/documents
```

12. Current Google Drive Backend Flow

```
User
↓
Django Drive endpoint
↓
Drive service
↓
Google Drive connection/auth context
↓
Drive synchronization
↓
Document records
↓
Celery/background processing
↓
Existing document ingestion pipeline
↓
Chunks + embeddings
↓
pgvector search
```

Google Drive is an external document source. Once a Drive document enters the backend processing pipeline, it follows the current document ingestion/indexing path.

13. What the Current Backend Already Has

- Django + Django REST Framework API backend.
- Authentication/verified-user access control.
- Document ownership checks.
- PDF, DOCX and TXT extraction.
- Background document processing with Celery.
- Retry handling for failed processing.
- Fixed word-based chunking with overlap.
- Sentence Transformer embeddings.
- PostgreSQL + pgvector.
- Vector similarity retrieval.
- Top-k source chunk retrieval.
- Page/chunk metadata.
- Groq answer generation.
- Conversation/chat history.
- Source chunks returned with answers.
- Similarity-based confidence score.
- Google Drive integration.
- Document update/reprocessing cleanup behavior.

14. What Is Not Yet in the Current Backend

- Structure-aware PDF extraction.

- Reliable table extraction + Markdown serialization.
- OCR/multimodal image processing path.
- Semantic chunking.
- Structure-aware chunking.
- Parent/child retrieval.
- Embedding cache.
- PII/access tags as a dedicated retrieval metadata system.
- Query guardrails.
- Query rewriting.
- Query decomposition.
- Semantic response cache.
- Hybrid vector + BM25 retrieval.
- Rich metadata filtering pipeline.
- Dedicated reranking stage.
- Parent expansion stage.
- Context compression.
- True groundedness verification.
- Formal citation generation layer.
- Agent planner/executor/tool architecture.
- Dedicated evaluation framework.
- Dedicated RAG observability/tracing pipeline.
- Retrieval tuning framework.

15. Current Backend Architecture — Simple Mental Model

```

Django API
↓
Services
↓
Celery (for background work)
↓
Document Processing
↓
Chunking
↓
Embeddings
↓
PostgreSQL + pgvector
↓
Similarity Retrieval
↓
Context Builder
↓
Groq LLM
↓
Answer + Sources + Confidence

```

16. Current vs Planned Backend Direction

Area	Current backend	Planned advanced backend
Ingestion	Basic extraction	Structure-aware + tables + OCR/multimodal

Area	Current backend	Planned advanced backend
Chunking	Fixed words + overlap	Semantic + structural + parent/child
Embeddings	Sentence Transformer	Embedding service + cache
Retrieval	Vector similarity	Hybrid vector + BM25 + filters + reranker
Query	Direct question	Guardrails + rewrite + decomposition
Context	Basic context	Parent expansion + compression
Generation	Groq + prompt	Generation + groundedness + citations + fallback
Agent	Not established in inspected backend	Planner + executor + tools + state/memory
Quality	Basic confidence heuristic	Evaluation + retrieval/generation metrics
Operations	Celery processing	Workers + observability + tracing + monitoring

17. Important Note About the Structure

This document intentionally describes only the current backend. It does not include frontend folders, frontend request flows, GitHub branch strategy, GitHub commit workflow, or the future advanced repository skeleton. The purpose is to provide a clean reference for understanding what the current backend actually does today.

18. Backend Responsibility Summary

- Views = receive HTTP requests and delegate work.
- Services = application/business orchestration.
- Tasks = asynchronous/background execution.
- Utilities = document parsing, chunking, embeddings and RAG generation helpers.
- Models = persistent database representation.
- Serializers = API validation/representation.
- Permissions = access-control rules.
- PostgreSQL/pgvector = persistence + vector retrieval.
- Celery = background processing.
- Redis = task broker/cache infrastructure.
- Groq = current answer-generation provider.